

The Enhanced UGV Cluster Navigation Methodology Based on MARL and LSTM

Yinjun Cheng
School of Computer and Science
Hubei University of Technology
Wuhan 430070, China
102111110@hbut.edu.cn

Siwei Wei*
CCCC Second Highway
Consultants Co. Ltd.
Wuhan 430056, China
weisiwei@ccccltd.cn

Zexi Chen
Xiaomi Technology (Wuhan)
Consultants Co., Ltd
Wuhan 430205, China
jerseychen998@gmail.com

Abstract—The main focus of this paper is to address the obstacle avoidance and path planning challenges in Unmanned Ground Vehicle (UGV) cluster navigation tasks within unknown environments of a certain scale, proposing a Multi-Agent Reinforcement Learning (MARL) based method for UGV cluster navigation. The approach leverages the MATD3 algorithm, which features an architecture with two sets of Critic networks to mitigate the issue of overestimation in reinforcement learning. Additionally, an LSTM network is integrated to address the inefficiency in utilizing time-series information in sample data, guiding UGV clusters to achieve global mission objectives without relying on explicit environmental maps. Furthermore, to expedite neural network training for UGV clusters in unknown environments, the paper employs Pybullet to simulate unknown environments. Subsequently, this paper compares this algorithm with MADDPG and MATD3 methods, demonstrating that our proposed approach outperforms navigation tasks within unknown environments. UGV clusters utilizing LSTM-MATD3 achieve higher cumulative rewards and success rates, along with lower cluster collision rates. Experimental results affirm the feasibility and effectiveness of our proposed method.

Keywords—deep reinforcement learning (DRL), multi-agent, UGV, cluster navigation

I. INTRODUCTION

In the navigation tasks of UGV clusters, they must engage in obstacle avoidance and path planning within the task environment. The objective is to determine an optimal path from the starting point to the target point while circumventing obstacles in the workspace based on certain criteria [1]. Traditional navigation methods such as Simultaneous Localization and Mapping (SLAM) [2], trajectory planning, and tracking control are typically applicable in static environments; however, adjustments are required when the environment changes. Consequently, recent years have witnessed a surge in research focused on local path planning, increasingly intertwining with fields such as neural networks and reinforcement learning. Chen et al. [3] addressed the issue of continuous action space and designed a Deep Deterministic Policy Gradient algorithm (DDPG) within the Actor-Critic (AC) framework. Lei et al. [4] proposed a reinforcement learning path planning theory based on the Double Deep Q Network (DDQN) algorithm. The continuous evolution of Multi-Agent Reinforcement Learning (MARL) enables agents to autonomously generate behaviour and alter their state information, ultimately achieving goals through effective collaboration with others [5]. Wei et al. [6] introduced the Multi-Agent Deep Deterministic Policy Gradient (MADDPG) algorithm, employing centralized training and decentralized execution (CTDE) [7] methods to enable agents to access the actions and observations of other agents during the training phase, including even global observations of the entire

environment. However, the aforementioned methods exhibit poor exploration capabilities in complex unknown environments, slow convergence rates, prolonged algorithm execution times, and susceptibility to local optima [8].

This study utilizes the Pybullet platform for simulation construction in unknown environments. UGVs can utilize laser radar for scanning within a certain range, obtaining distance information to target task points, and generating velocity commands for UGVs. Subsequently, UGVs are trained iteratively based on their policies [9]. When the training policy reaches optimality, UGVs can engage in path planning and obstacle avoidance accordingly. Herein, UGVs adopt the Multi-Agent Temporal Difference Deep Deterministic Distributional Policy Gradient (MATD3) [10] algorithm, employing two sets of Critic networks to address the issue of overestimation in reinforcement learning. Since UGVs perceive the environment partially, actions taken based on the policy possess temporal characteristics. If historical observation information is disregarded and decisions are solely based on current observations, decision conflicts may arise. Leveraging the capability of Long Short-Term Memory (LSTM) in handling time-series data, this paper introduces the LSTM-MATD3 algorithm, thereby enhancing the UGV cluster's capability in obstacle avoidance and path planning in unknown environments. Furthermore, it brings the UGV cluster closer to the core objectives of improved performance, pattern discovery, and environmental adaptation.

II. PROBLEM FORMULATION

A. Environment Setup

This paper addresses the rapid completion of UGV cluster navigation tasks in unknown environments of a certain scale [11]. The task requires UGVs to autonomously explore, respond to, and navigate obstacles based on the characteristics of the unknown environment, as well as the ability to autonomously reach target task points in real time [12]. As illustrated in Fig. 1, we create a scenario based on the Pybullet platform, consisting of 3 blue UGVs and 3 red target task points within a task area of 10x10 units, with 15 black obstacles. The starting points, target task points, and obstacle positions for each UGV are randomly generated in each round to simulate unknown environments for training and testing purposes. Each UGV utilizes laser radar for environmental detection, obtaining information on task target points, obstacles, and the positions of other UGVs, while autonomously navigating to the target location and avoiding obstacles.

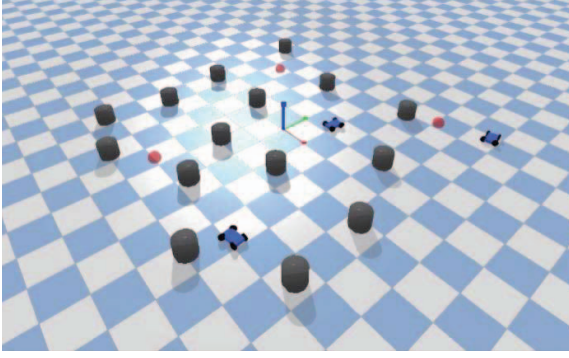


Fig. 1. Simulation environments for UGV cluster training.

B. State Space Design

In a UGV cluster, the number of UGVs is denoted by N . The state of an individual UGV, represented as O_t , comprises two components: the UGV's state, denoted as O_{self} , and the environmental state, denoted as O_{env} , defined as follows in Equation 1.

$$O_t = (O_{self}, O_{env}) \quad (1)$$

The self-state of a UGV includes the following components: the instantaneous linear velocity vector at the current time, denoted as $v = (v_x, v_y)$, the instantaneous angular velocity, denoted as ω , the yaw angle, denoted as ψ , and the difference between the UGV's position and the target position in the simulation environment, denoted as $P = (P_x, P_y)$. Therefore, the self-state of the UGV is defined as follows in Equation 2.

$$O_{self} = (v, \omega, \psi, P) \quad (2)$$

The environmental state is defined as follows in Equation 3.

$$O_{env} = (\Phi_1, \Phi_2, \dots, \Phi_N) \quad (3)$$

The simulation environment in this paper utilizes a 24-beam 2D laser radar [13], denoted as $\Phi = (\phi_1, \phi_2, \dots, \phi_{24})$. The detection range of the laser radar sensor is limited from 0.3 m to 3 m. The laser radar enables real-time acquisition of the positions of other UGVs, obstacles, and target task points.

C. Action Space Design

The UGV adheres to a non-holonomic constrained kinematic model. Referring to the control dimensions of the agent's kinematic model, the action space is designed for it, transformed into a first-order system. In this paper, the action space for each UGV is defined as $A = (v_{UGV}, \psi)$, encompassing the UGV's current linear velocity v_{UGV} and steering wheel yaw angle ψ at a certain moment. The linear velocity of the steering wheel is constrained from 0 m/s to 1 m/s, and the steering wheel yaw angle is constrained from -1 rad to 1 rad.

D. Reward Function Design

In multi-task path planning, the UGV cluster aims to select the shortest possible path after completing tasks, reflecting an implicit cooperative relationship where multiple UGVs complete an overall task without direct competition. In experiments, when one UGV completes visiting a target task

point, all UGVs receive the same positive reward value. This incentivizes all UGVs to cooperate in completing the visit to the target point and prevents situations where two UGVs compete for the same target task point.

Based on the above, to ensure that each UGV successfully reaches the target task point while considering obstacle avoidance and selecting the optimal path, the total reward function for each UGV is defined as follows in Equation 4.

$$r_{total} = k_1 r_{\omega i}(t) + k_2 r_{\phi i}(t) + k_3 r_{\delta i}(t) \quad (4)$$

Where k_1 , k_2 , and k_3 represent weighting coefficients, with values of 0.5, 2, and 5 respectively. The velocity reward function $r_{\omega i}(t)$, arrival reward function $r_{\phi i}(t)$, and collision penalty function $r_{\delta i}(t)$ are defined as follows in Equations 5, 6 and 7 respectively.

1) Velocity reward function $r_{\omega i}(t)$:

$$r_{\omega i}(t) = \frac{avgD_{t-1} - avgD_t}{MaxV} \quad (5)$$

This reward incentivizes UGVs to explore the expected trajectory at a certain velocity. $avgD_t$ represents the average distance difference between UGV and each effective target task point at the current time t , while D_{t-1} represents the average distance difference at the previous time step $t-1$. $MaxV$ denotes the maximum linear velocity of the unmanned vehicle.

2) Arrival reward function $r_{\phi i}(t)$:

$$r_{\phi i}(t) = \begin{cases} +1, & \text{if } g_j \text{ has been visited} \\ -W \times avgD_t, & \text{if } g_j \text{ has been unvisited} \end{cases} \quad (6)$$

This reward provides an immediate reward value for each step taken by the UGV in a single experiment, thereby addressing the sparse reward problem before the overall task is completed. Here, $g_j, j(j = N)$ represents the target task point, and if it is visited, the UGV receives a reward of +1. Otherwise, the reward value is determined based on the magnitude of $avgD_t$. W is an adjustment weight, where a value of 0.01 is chosen.

3) Collision penalty function $r_{\delta i}(t)$:

$$r_{\delta i}(t) = \begin{cases} -2, & \text{if } \|g_i - g_j\| \leq D_i^\epsilon \\ -1, & \text{if } \|g_i - g_j\| \leq D_i^* \\ 0, & \text{not collision} \end{cases} \quad (7)$$

This reward carries a punitive aspect, indicating whether collisions occur between two UGVs or between a UGV and an obstacle. A safety distance is introduced between UGVs and obstacles by setting a safe distance D_i^* , which is the sum of the radii of the key regions D_i^ϵ of the two objects. When the key regions of two objects overlap, it is termed as a pseudo-collision. Similarly, if the distance between UGVs or between a UGV and an obstacle is less than the key region, it is termed as a real collision.

When a pseudo-collision occurs between two objects, the UGV receives a negative reward of -1. If a real collision occurs, the reward is -2, and the current round ends. If no collision occurs, the reward is 0. Through this collision warning mechanism, pseudo-collisions are penalized to reduce the occurrence of real collisions, ensuring the stability of the UGV learning process.

III. THE LSTM-MATD3 ALGORITHM

In reinforcement learning, the action-value function is maximized and used to learn the optimal policy of the agent.

However, the resulting high estimation bias often leads to inaccurate learned policies [14]. LSTM-MATD3 employs two sets of actor-critic structures, with each agent having its independent network. The actor network $\mu(o_i; \theta_i^\mu)$ generates actions a_i based on the current local observations o_i of the UGV r_i , where the network parameters are denoted as θ_i^μ . Let x denote the joint observation values of all UGVs, represented as $x = (o_1, o_2, \dots, o_N)$.

Two critic networks, $Q_1(x, a_1, \dots, a_N; \theta_i^{Q_1})$ and $Q_2(x, a_1, \dots, a_N; \theta_i^{Q_2})$, are trained in a centralized manner. They take the observations and actions of all UGVs as inputs and estimate the Q-values as outputs, evaluating the goodness of action a_i to enhance the performance of the actor network. The network parameters are denoted as $\theta_i^{Q_1}$ and $\theta_i^{Q_2}$. To ensure the stability of UGV policy learning, target actor and two target critic networks are constructed using parameters $\hat{\theta}_i^\mu, \hat{\theta}_i^{Q_1}$ and $\hat{\theta}_i^{Q_2}$. The update mechanism for the target networks is defined as follows in Equations 8 and 9.

$$\hat{\theta}_i^\mu = \tau \theta_i^\mu + (1 - \tau) \hat{\theta}_i^\mu \quad (8)$$

$$J(\theta_i^{Q_j}) = \mathbb{E}_{o^t, a^t, r^t, o^{t+1}} \left[\left(Q_j(x^t, a_1^t, \dots, a_N^t; \theta_i^{Q_j}) - y_i^t \right)^2 \right]; j = 1, 2 \quad (9)$$

The parameters $\theta_i^{Q_1}$ and $\theta_i^{Q_2}$ of the two critic networks are updated via gradient descent, as shown in Equation 13.

$$\theta_i^{Q_j} = \theta_i^{Q_j} - \alpha_c^j \nabla_{\theta_i^{Q_j}} J(\theta_i^{Q_j}); j = 1, 2 \quad (13)$$

$$\nabla_{\theta_i^\mu} J(\theta_i^\mu) = \mathbb{E}_{o^t, a^t} \left[\nabla_{\theta_i^\mu} \mu_i(o_i^t; \theta_i^\mu) \nabla_{a_i} Q(o_1^t, \dots, o_N^t, a_1^t, \dots, a_N^t; \theta_i^Q) \Big|_{a_i = \mu_i(o_i^t; \theta_i^\mu)} \right] \quad (14)$$

The network parameters of the actor network are updated through stochastic gradient ascent (SGA).

LSTM neural networks possess the capability of long-term memory, enabling them to predict outcomes based on historical data and current inputs. This ability allows LSTM networks to address decision conflicts under partial observation conditions, allowing UGVs to take different actions based on previous observation information [15]. In the MATD3 algorithm, the actor network takes the observation of a single UGV on the environment as input. Compared to the centralized critic network, the actor network's handling of time-series data is more frequent and direct [16]. Therefore, we attempt to introduce LSTM into the actor network to observe whether its performance in UGV cluster navigation tasks can be improved. In this study, the actor network with LSTM takes actions based on LSTM hidden elements h_i and current observation o_i , rather than solely on the current observation. The process by which UGVs take actions is defined as shown in Equation 15.

$$a_i = \mu(o_i, h_i; \theta_i^\mu) + \text{clip}(N(0, \sigma), -c_i, c_i) \quad (15)$$

IV. PERFORMANCE ANALYSIS

To verify the effectiveness of the LSTM-MATD3 algorithm in solving path-planning problems in unknown environments, simulations were conducted. As shown in Fig. 2, in the simulation scenario, the UGV cluster not only avoids environmental obstacles but also exhibits avoidance behaviour between UGVs. Eventually, all UGVs successfully

$$\hat{\theta}_i^{Q_j} = \tau \theta_i^{Q_j} + (1 - \tau) \hat{\theta}_i^{Q_j}; j = 1, 2 \quad (9)$$

Where τ is the soft update parameter. To alleviate the problem of overestimation bias, it is common practice to select the smaller Q-value from the two critic networks to compute the target Q-value, thereby enhancing the accuracy of action value evaluation by the critic networks. The target Q-value for UGV r_i is defined as follows in Equation 10.

$$y_i^t = r_i^t + \gamma \min \left(Q_j(x^{t+1}, \tilde{a}_1^{t+1}, \dots, \tilde{a}_N^{t+1}; \hat{\theta}_i^{Q_j}) \right); j = 1, 2 \quad (10)$$

Where γ is the reward discount factor, r_i^t is the reward obtained by the current interaction between UGV r_i and the environment. $\tilde{a}_i^{t+1}$ is the action output by the target actor network for UGV i based on the agent's observation, as shown in Equation 11.

$$\tilde{a}_i^{t+1} = \mu(o_i^{t+1}; \hat{\theta}_i^\mu) + \text{clip}(N(0, \sigma), -c_i, c_i) \quad (11)$$

Where $N(0, \sigma)$ represents random noise following a Gaussian distribution, and c_i is the boundary value of the noise. The loss functions $J(\theta_i^{Q_1})$ and $J(\theta_i^{Q_2})$ for the two critic networks of UGV r_i are defined as shown in Equation 12.

Where α_c^1 and α_c^2 are the learning rates for the two critic networks. The gradient computation formula for the actor network of UGV r_i is defined as shown in Equation 14.

searched for and reached the target task points. The trajectories of each UGV are represented by different colours.

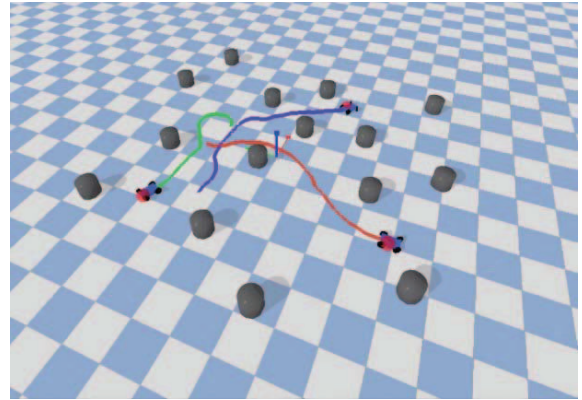


Fig. 2. UGV Cluster Experiment in the Simulation Environment.

The main algorithm used for performance evaluation in this study is LSTM-MATD3, with MADDPG and MATD3 as the comparative algorithms. The experimental results are illustrated in Figs. 3 to 5, where the x-axis represents the training steps of the UGV cluster, and the y-axis represents the cumulative reward, success rate, and collision rate of the UGV cluster, respectively.

Fig. 3 displays the cumulative reward values of the three algorithms. A higher cumulative reward indicates a better mastery of the environment and reward function by the UGV cluster. LSTM-MATD3 outperforms MADDPG and MATD3

in terms of cumulative reward. This implies that the LSTM-MATD3 algorithm can better coordinate the behavior of the UGV cluster, optimize its behavioral strategies, and thereby maximize the overall system reward.

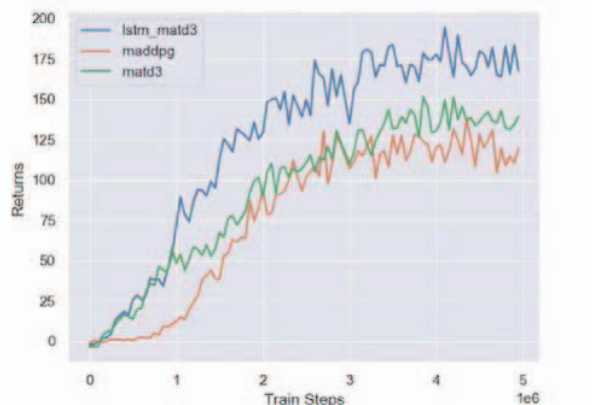


Fig. 3. Cumulative Rewards of the UGV Cluster.

The success rate refers to the absence of collisions within a specified number of steps for the UGV cluster, which is considered successful. It is one of the indicators used to evaluate the efficiency and convergence speed of multi-agent algorithms. As depicted in Fig. 4, LSTM-MATD3 consistently exhibits a higher success rate compared to MADDPG and MATD3. This suggests that LSTM-MATD3 is more effective in achieving the specified goals when handling multi-UGV collaborative tasks.

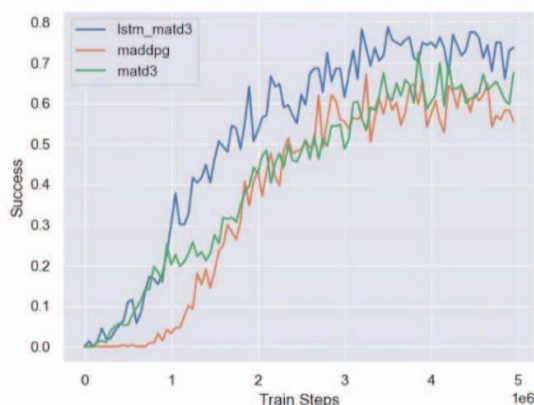


Fig. 4. Success Rate of the UGV Cluster.

Fig. 5 illustrates the collision situations of the three algorithms in two sets of experiments. Once a real collision occurs between UGVs or with obstacles, the current round is immediately terminated. As the algorithms converge, LSTM-MATD3 exhibits a collision rate similar to MADDPG, while MATD3 shows a lower collision rate but with weaker obstacle avoidance capability, requiring fewer steps. Overall, LSTM-MATD3 maintains reasonable performance, enabling the system to converge quickly. In summary, LSTM-MATD3 can achieve convergence with fewer steps and effectively avoid collisions in multi-agent systems, thereby enhancing system safety and reliability.

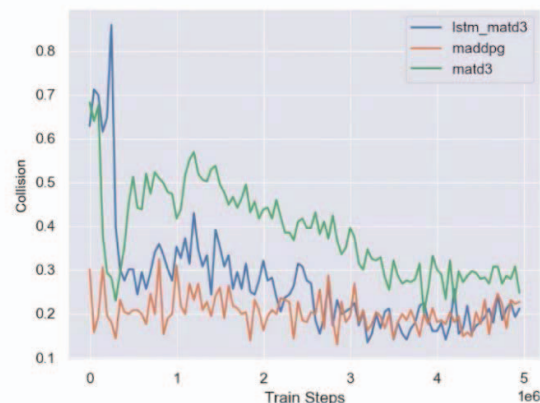


Fig. 5. Collision Rate of the UGV Cluster.

V. CONCLUSION

The paper primarily discusses two approaches: multi-agent reinforcement learning and LSTM networks, and their combination and enhancements. In the absence of maps, our method enables UGV clusters to navigate from arbitrary starting points to specified destinations in unknown environments without collisions. Results demonstrate the excellent performance of the LSTM-MATD3 algorithm in UGV clusters, with varying degrees of improvement in reward accumulation, success rate, and collision rate, indicating its potential for collaborative decision-making tasks. Although performance may degrade when scaling to larger UGV clusters, it still holds certain advantages, leaving room for future enhancements. For instance, introducing moving obstacles or further increasing the number of UGV clusters to simulate more complex environmental factors could be explored in future research.

REFERENCES

- [1] Xie, R., Meng, Z., Wang, L., Li, H., Wang, K., et al. (2021). Unmanned aerial vehicle path planning algorithm based on deep reinforcement learning in large-scale and dynamic environments. *IEEE Access*, 9, 24884-24900
- [2] Wen, S., Zhao, Y., Yuan, X., Wang, Z., Zhang, D., et al. (2020). Path planning for active SLAM based on deep reinforcement learning under unknown environments. *Intelligent Service Robotics*, 13, 263-272
- [3] Chen, Y., Han, W., Zhu, Q., Liu, Y., & Zhao, J. (2022). Target-driven obstacle avoidance algorithm based on DDPG for connected autonomous vehicles. *EURASIP Journal on Advances in Signal Processing*, 2022 (1), 1-22
- [4] Lorberbom, G., Maddison, C. J., Heess, N., Hazan, T., & Tarlow, D. (2020). Direct policy gradients: Direct optimization of policies in discrete action spaces. *Advances in Neural Information Processing Systems*, 33, 18076-18086
- [5] Wang, C., Cheng, C., Yang, D., Pan, G., & Zhang, F. (2022). Path Planning in Localization Uncertain Environment Based on Dijkstra Method. *Frontiers in Neurorobotics*, 16, 821991
- [6] Wei, X., Yang, L., Cao, G., Lu, T., & Wang, B. (2020). Recurrent MADDPG for object detection and assignment in combat tasks. *Ieee Access*, 8, 163334-163343
- [7] Li, W., Yue, M., Shanguan, J., & Jin, Y. (2023). Navigation of Mobile Robots Based on Deep Reinforcement Learning: Reward Function Optimization and Knowledge Transfer. *International Journal of Control, Automation and Systems*, 21 (2), 563-574
- [8] Zheng, J., Mao, S., Wu, Z., Kong, P., & Qiang, H. (2022). Improved path planning for indoor patrol robot based on deep reinforcement learning. *Symmetry*, 14 (1), 132
- [9] Rahim, S., Razaq, M. M., Chang, S. Y., & Peng, L. (2022). A reinforcement learning-based path planning for collaborative UAVs

- Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing, 1938-1943.
- [10] Zhou, C., Li, J., Shi, Y., & Lin, Z. (2023). Research on Multi-Robot Formation Control Based on MATD3 Algorithm. *Applied Sciences*, 13 (3), 1874
 - [11] Luis, S. Y., Reina, D. G., & Marín, S. L. T. (2021). A multiagent deep reinforcement learning approach for path planning in autonomous surface vehicles: The Ypacaraí lake patrolling case. *IEEE Access*, 9, 17084-17099
 - [12] Ali, N., Kamarudin, K., Bakar, M. A. A., Rahiman, M. H. F., Zakaria, et al. (2023). 2D LiDAR based Reinforcement Learning for Multi-target Path Planning in Unknown Environment. *IEEE Access*
 - [13] Beomsoo, H., Ravankar, A. A., & Emaru, T. (2021). Mobile robot navigation based on deep reinforcement learning with 2d-lidar sensor using stochastic approach 2021 IEEE International Conference on Intelligence and Safety for Robotics (ISR), 417-422: IEEE.
 - [14] Bayerlein, H., Theile, M., Caccamo, M., & Gesbert, D. (2021). Multi-UAV path planning for wireless data harvesting with deep reinforcement learning. *IEEE Open Journal of the Communications Society*, 2, 1171-1187
 - [15] Wang, W., Wu, Z., Luo, H., & Zhang, B. (2022). Path planning method of mobile robot using improved deep reinforcement learning. *Journal of Electrical and Computer Engineering*, 2022, 1-7
 - [16] Grando, R. B., de Jesus, J. C., Kich, V. A., Kolling, A. H., & Drews-Jr, P. L. J. (2022). Double critic deep reinforcement learning for mapless 3d navigation of unmanned aerial vehicles. *Journal of Intelligent & Robotic Systems*, 104 (2), 29